
Willow - Know whats really in it

MVP Project Proposal

An AI-based product ingredient scanner for the Indian market

Flagging harmful ingredients and suggesting healthier alternatives from a single product photograph



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Course: Software Engineering (UCS 503)
Team: Aditya Gupta 1024160001
agupta6be24@thapar.edu Akanksha
Thakur 1024160019
athakur3be24@thapar.edu
Domain: Computer Vision, NLP, Full-Stack Development

Contents

1 Introduction	1
2 Motivation	2
3 Problem Statement	2
4 Proposed Solution	3
4.1 Handling products with no ingredient list	3
4.2 Design principle: built to extend	3
5 MVP Scope	4
5.1 In scope	4
5.2 Out of scope for MVP (future work)	4
6 Functional Requirements	4
7 Non-Functional Requirements	5
8 System Architecture	6
8.1 Data requirements	6
8.2 Recommended technology stack	6
9 Data Sources	7
10 Success Criteria	7
11 Conclusion	7

1 Introduction

Packaged and locally sold food products in India frequently carry ingredient information that is incomplete, inconsistent, or entirely absent from a consumer's point of view. Rising health consciousness among Indian consumers has increased demand for tools that explain what is actually inside a product, yet the tools that exist today are not built for this market.

This proposal outlines a Minimum Viable Product (MVP) for an Ingredient Scanner App Willow: a system that lets a user photograph a packaged food product and receive an analysis of its ingredients which ones are potentially harmful, what health effects they are associated with, and what healthier

alternatives exist in the same category and price range. The system is designed around image-based extraction rather than barcode lookup, so it can meaningfully cover local and unbranded Indian products that barcode-database-driven tools cannot.

2 Motivation

Three factors motivate this project:

- A documented food safety problem. Indian regulatory bodies have repeatedly intercepted adulterated food products in recent years, including large seizures of adulterated spices, and Indian spice brands have faced bans abroad over pesticide contamination. This reflects a real, ongoing information gap between what consumers buy and what they know about it.
- Rising health awareness with no matching tool. Indian consumers increasingly want to make informed choices about food and personal care products, but existing ingredientscanning apps are calibrated to Western labeling standards, additive lists, and barcodeindexed brands.
- A structural gap in existing tools. Global scanning apps rely on barcode-to-database lookups. This approach systematically fails for unbranded, regional, and small-vendor Indian products a large share of the market because these products are rarely barcode-indexed in the first place.

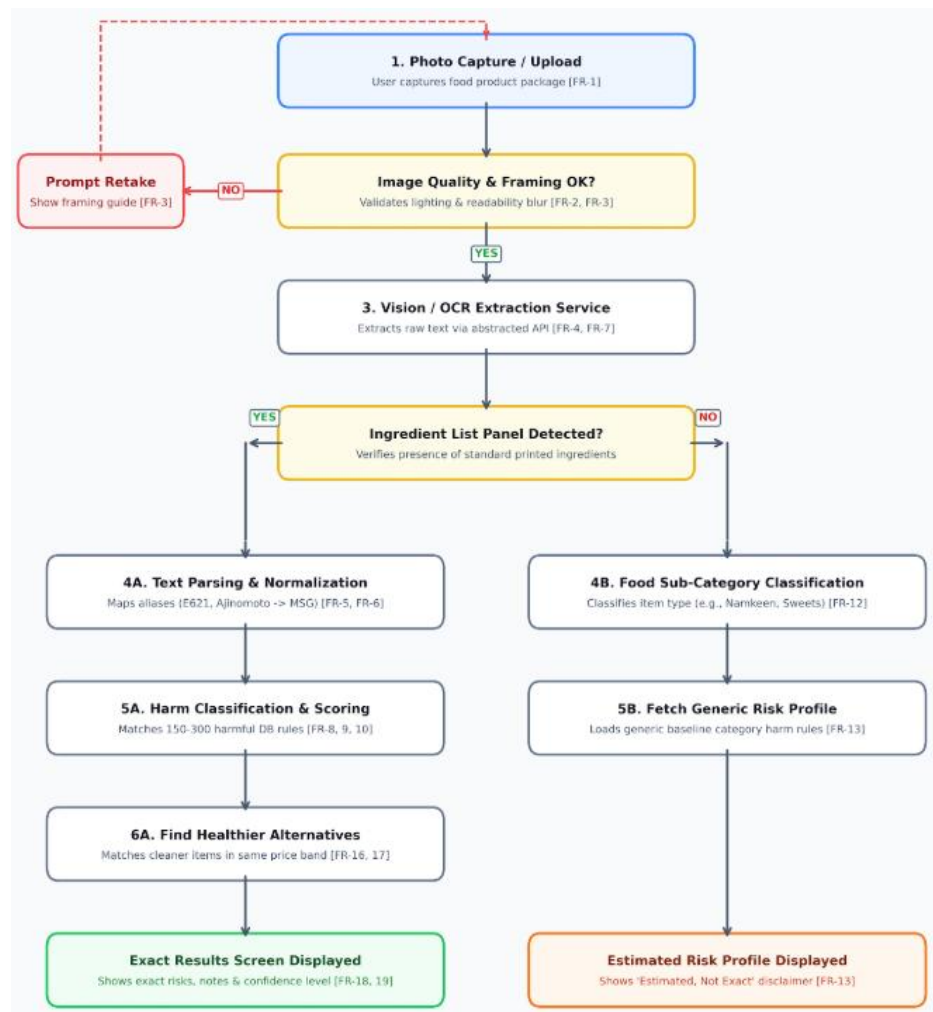
3 Problem Statement

Existing ingredient-scanning solutions available to Indian consumers are inadequate for three concrete reasons:

1. Barcode dependency excludes most local products. Apps such as Yuka rely on barcode databases populated primarily with branded, internationally distributed products, leaving unbranded and regional Indian products unscanned.
2. Many local products carry no printed ingredient list at all. Local sweets, bakery items, and loose-packaged snacks are common in the Indian market and frequently have no ingredient panel to read, which barcode- or OCR-only approaches cannot handle.
3. Scoring standards are not localized. Existing tools apply EU/US additive standards rather than FSSAI guidelines, and do not account for allergens or ingredients (e.g., Ayurvedic/traditional ingredients) specific to the Indian market.

There is, at present, no widely available tool that combines image-based (non-barcode) ingredient extraction with India-specific harmful-ingredient scoring and locally relevant alternative suggestions.

4 Proposed Solution



4.1 Handling products with no ingredient list

When no ingredient list can be extracted a common case for unbranded and local Indian products the system falls back to a category-level generic risk profile rather than failing or guessing. The product is classified into a known food sub-category (e.g., namkeen, bakery, sweets) using available text and visual cues, and a generic, clearly labeled "estimated, not exact" risk summary is shown instead of a fabricated exact analysis.

4.2 Design principle: built to extend

Although the MVP is scoped to a single product category (food), the system is architected so that category-specific behavior ingredients, harm rules, and alternatives is driven by data and configuration rather than hard-coded logic. This means additional categories (e.g., cosmetics, spices as a flagship sub-case) and additional languages can be added later without redesigning the core extraction, classification, or alternatives modules.

5 MVP Scope

5.1 In scope

- Single product category: packaged food, including packaged spices/masalas as a flagship sub-case.
- English-language ingredient labels (Hindi as a stretch goal).
- Photo-based ingredient extraction with no barcode dependency.
- A curated harmful-ingredients database covering approximately 150–300 common ingredients and additives.
- Category-level generic risk profiling as a fallback for unbranded/local products.
- Rule-based healthier-alternative suggestions.

5.2 Out of scope for MVP (future work)

- Additional categories: cosmetics/personal care, oral care, household cleaning, supplements.
- Crowdsourced ingredient database contributions at scale.
- FSSAI license number lookup and verification (FoSCoS integration).
- Regional-language OCR support beyond English.
- Visual product-matching for previously seen unbranded products.

6 Functional Requirements

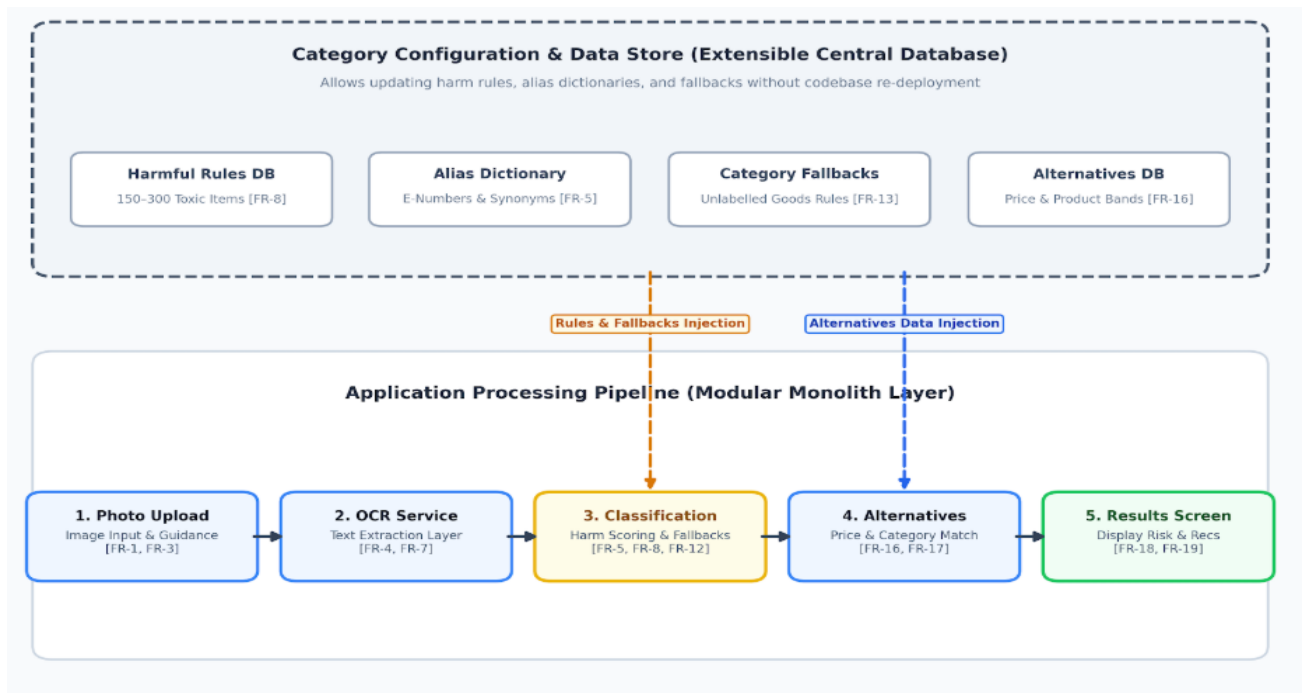
ID	Requirement	Priority
FR-1	User can capture a product photo using the device camera or upload an existing image.	Must-have
FR-2	App provides basic capture guidance (lighting, framing) to improve extraction accuracy.	Should-have
FR-3	App validates image quality and prompts a retake if extraction confidence is low.	Should-have
FR-4	System extracts raw text from the ingredient panel using an OCR/vision API.	Must-have
FR-5	System parses raw extracted text into a clean, structured ingredient list.	Must-have
FR-6	System normalizes ingredient name variants and aliases to a single canonical entity.	Must-have
FR-7	Extraction module is abstracted behind an internal interface so the OCR/vision provider can be swapped.	Must-have
FR-8	System checks each extracted ingredient against the harmful-ingredients database.	Must-have
FR-9	System displays flagged ingredients with a plain-language explanation of health effects.	Must-have
FR-10	System assigns an overall product risk indicator based on flagged ingredients.	Should-have

FR-11	Classification logic reads category-specific rules from configuration/data, not hard-coded per category.	Must-have
FR-12	If no ingredient list is detected, system attempts to classify the product into a known food sub-category.	Must-have
FR-13	System shows a generic, category-level risk profile with a clear "estimated, not exact" disclaimer.	Must-have
FR-14	If a visible FSSAI license number is detected, system surfaces it as a trust signal.	Could-have
FR-15	User can manually enter known ingredients when no list or category match is found.	Should-have
FR-16	System suggests alternative products in the same category with fewer/no flagged ingredients.	Must-have
FR-17	Suggested alternatives are matched within a comparable price range.	Should-have
FR-18	App displays extracted ingredients, flags, health-effect notes, and alternatives on one results screen.	Must-have
FR-19	App clearly indicates the confidence level of the analysis.	Must-have

7 Non-Functional Requirements

Category	Requirement
Scalability & Extensibility	New product categories can be added via data/configuration changes, without modifying the extraction, classification, or alternatives modules.
Performance	End-to-end analysis should complete within a few seconds under normal conditions; extraction calls run asynchronously rather than blocking the UI.
Accuracy & Reliability	Extraction confidence is surfaced to the user; low-confidence results prompt a retake or manual fallback.
Data Integrity	Harmful-ingredient claims are traceable to a source in the database; the system avoids unattributed or invented health claims.
Security & Privacy	Uploaded images and user-submitted data are stored securely; no unnecessary personal data is collected.
Usability	The core flow (photo → result) is usable without instructions; results are presented in plain, non-technical language.

8 System Architecture



8.1 Data requirements

- Categories table: holds category definitions (MVP: food only; schema supports more).
- Ingredients table: approx. 150–300 curated entries for MVP, each linked to a category, with associated health-effect notes.
- Aliases table: maps ingredient name variants (E-numbers, common/regional names) to a single canonical ingredient.
- Alternatives table: healthier product suggestions linked to category and price band.
- Generic risk profiles: category-level fallback data for unbranded-product classification.

8.2 Recommended technology stack

Layer	Recommendation
Ingredient extraction	Vision-capable API (multimodal model or OCR service) behind an internal abstraction layer
Backend	Modular monolith (e.g., Node.js/Express or Python/FastAPI)
Database	Relational database (e.g., PostgreSQL) for category/ingredient/alias data
Layer	Recommendation
Async processing	Lightweight job queue for extraction calls (e.g., BullMQ or Celery)

Frontend	Mobile-first web app or native app for photo capture and results display
Image storage	Object storage for uploaded photos

9 Data Sources

The harmful-ingredients database will be built by combining the following sources rather than relying on a single existing dataset:

- Open Food Facts — a crowdsourced, openly licensed database of food products with ingredients, additives, and allergen data, downloadable as CSV/JSONL/Parquet and filterable by country.
- FSSAI regulatory publications — the Compendium on Food Safety and Standards (Food Additives) Regulation, used as the authoritative source for India-specific permitted-additive rules; requires parsing from PDF into structured form.
- Existing ML-ready datasets (e.g., Kaggle ingredient/allergen datasets) — used to bootstrap the classification schema before refinement with India-specific data.

10 Success Criteria

- A user can photograph a real packaged food product and receive an accurate, structured ingredient list.
- Harmful ingredients present are correctly flagged with a clear, correct health-effect explanation.
- Unbranded/local products with no ingredient list still return a useful, clearly-labeled generic risk profile.
- At least one relevant, price-comparable healthier alternative is suggested where applicable.
- Adding a second category (as a demonstration exercise) requires only data/configuration changes, not pipeline rewrites.

11 Conclusion

The Ingredient Scanner App addresses a genuine and documented gap in the Indian consumer product market: the absence of a scanning tool built around Indian labeling realities, regulatory standards, and the prevalence of unbranded local products. By scoping the MVP to a single, well-defined category while architecting the system to be data-driven and extensible, this project is achievable within a one-semester timeline for a team of 2–3, while laying a credible foundation for future expansion into additional product categories and markets.